Fraud

Assignment

For **Spring 2026**, the **Fraud** lab will be used as **Lab 03**.

> Before submission, especially of your report, you should be sure to review the **Lab Policy** page.

- **PrairieLearn**: Lab *Model*
- **PrairieLearn**: Lab *Report*

> On Canvas, be sure to submit both your source `.ipynb` file and a rendered `.html` version of the report.

- The **model** portion of the lab is due on Saturday, **March 28**.
- The **report** portion of the lab is due on Saturday, **April 04**.

Background

Every day, millions if not billions of credit card transactions are processed.

- Wikipedia: Credit Card

Banks and credit card companies face a constant battle against fraudulent transactions.

- Consumer Financial Protection Bureau: Fraud and Scams

Fraudsters continuously develop new tactics to steal money, from card skimming to account takeovers to testing stolen cards with small purchases. To a consumer, a transaction is processed quickly, but behind the scenes, sophisticated systems are used to determine whether the transaction is legitimate or fraudulent.

Detecting fraud in real-time is remarkably difficult for several key reasons:

- *Asymmetric Costs*: The costs of false negatives and false positives, from both the perspective of the banks and the customers, must be carefully considered.
- *Extreme Rarity*: Fraudulent transactions are exceptionally rare. This extreme class imbalance makes fraud difficult to detect, as naive models can achieve high accuracy by simply predicting "not fraud" for all transactions.
- *Speed Requirements*: Decisions must happen in milliseconds while a customer waits at checkout.

Modern fraud detection relies on machine learning models that analyze transaction characteristics in real-time, score fraud likelihood, and automatically make approval decisions. This lab will give you hands-on experience with the core challenge: building a classifier that appropriately balances catching fraud while avoiding false alarms.

Scenario and Goal

Who are you?

- You are a data scientist working for a banking institution that issues credit cards to their customers.

What is your task?

- You are tasked with creating an automated fraud detector. As soon as a credit card transaction is made, given the information available at the time of the transaction (location, amount, etc), your model should immediately identify the transaction as fraudulent or genuine. Your **goal** is to find a model that appropriately balances false positives and false negatives.

Who are you writing for?

- To summarize your work, you will write a report for your manager, who is the head of the loss minimization team. You can assume your manager is very familiar with banking and credit cards, and reasonably familiar with the general concepts of machine learning.

Data

To achieve the goal of this lab, we will need information on previous credit card transactions, including whether or not they were fraudulent. The necessary data is provided in the following files:

- Train Data: `fraud-train.parquet`
- Test Data: `fraud-test.parquet`

Source

The data for this lab originally comes from Kaggle. Citations for the data can be found on Kaggle.

- **Kaggle**: Credit Card Fraud Detection

A brief description of the target variable is given.

> This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

Similarly, a brief description of the feature variables is given.

> It contains only numerical input variables which are the result of a PCA transformation. Unfortunately, due to confidentiality issues, we cannot provide the original features and more background information about the data. Features V1, V2, ... V28 are the principal components obtained with PCA, the only features which have not been transformed with PCA are 'Time' and 'Amount'. Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset. The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise.

We are providing a modified version of this data for this lab.

Modifications include:

- Removed the `Time` variable as it is misleading.
- Reduced the number of samples, while maintaining the number of fraudulent transactions.
  - The class imbalance is reduced, but the target is still highly imbalanced.
- Withheld some data that will be considered the production data.
- Renamed the target variable from `Class` to `Fraud`.
- Renamed the PCA transformed variables.

Data Dictionary

Each observation in the train, test, and (hidden) production data contains information about a particular credit card transaction.

The variable descriptions listed below are available in the Markdown file `variable-descriptions.md` for ease of inclusion in reports.

Variable Descriptions

`Fraud`

- `[int64]` status of the transaction. `1` indicates a fraudulent transaction and `0` indicates not fraud, a genuine transaction.

`Amount`

- `[float64]` amount (in dollars) of the transaction.

`PC01 - PC28`

- `[float64]` the 28 principal components that encode information such as location and type of purchase while preserving customer privacy.

> Principal Component Analysis (**PCA**) is a method that we will learn about later in the course. For now, know that it takes some number of features as inputs, and outputs either the same or fewer features, that retain most of the original information in the features. You can assume things like location and type of purchase were among the original input features. (Ever had a credit card transaction denied while traveling?)

Data in Python

To load the data in Python, use:

```
import pandas as pd
```

```
fraud_train = pd.read_parquet(
    "https://lab.cs307.org/fraud/data/fraud-train.parquet",
)
fraud_test = pd.read_parquet(
    "https://lab.cs307.org/fraud/data/fraud-test.parquet",
)
```

Prepare Data for Machine Learning

Create the `X` and `y` variants of the data for use with `sklearn`:

```
# create X and y for train
X_train = fraud_train.drop("Fraud", axis=1)
y_train = fraud_train["Fraud"]

# create X and y for test
```

```
X_test = fraud_test.drop("Fraud", axis=1)
y_test = fraud_test["Fraud"]
```

You can assume that within the autograder, similar processing is performed on the production data.

Sample Statistics

Before modeling, be sure to look at the data. Calculate the summary statistics requested on PrairieLearn.

Models

To obtain the maximum points via the autograder, your submitted model must outperform the following metrics:

- Test Precision: 0.9
- Test Recall: 0.8
- Production Precision: 0.9
- Production Recall: 0.8